



北京航空航天大学
BEIHANG UNIVERSITY



数据结构与程序设计（信息类）

Data Structure & Programming

北京航空航天大学 数据结构课程组

软件学院 谭火彬

2024年春



提纲：绪论

- ◆ 0.1 课程简介
- ◆ 0.2 数据结构基础
- ◆ 0.3 算法基础



提纲：绪论

◆ 0.1 课程简介

- ✓ 0.1.1 课程资源

- ✓ 0.1.2 课程安排

◆ 0.2 数据结构基础

◆ 0.3 算法基础



课程资源

◆教师

- ✓谭火彬，软件学院)
- ✓Email: thbin@buaa.edu.cn

◆助教

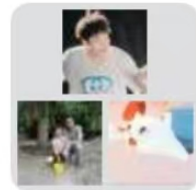
- ✓曹之龙、吴奇萱

◆上机

- ✓J5-401，周四晚上20:35-22:15

◆考核方式

- ✓作业：20%，期中：30%，期末：50%
- ✓作业及考试均采用上机方式（在线考试）



群聊：课程2024春-数据结构与程序设计



该二维码7天内(3月5日前)有效，重新进入将更新



教材和参考书

- ◆ 《数据结构与程序设计》（第3版），唐发根，北京航空航天大学出版社，2017
- ◆ 《C程序设计导引-原理与实践》，宋友、王君臣、肖文磊、李莹、谭火彬编著，高等教育出版社，2022
- ◆ 参考资料
 - ✓ 《C程序设计导引》，尹宝林，机械工业出版社，2017
 - ✓ 《数据结构与算法分析-C语言描述》，M.A.Weiss著 冯舜玺译，机械工业出版社，2014
 - ✓ 《C++数据结构与算法分析(第4版)》，M.Drozdek著 徐丹 吴伟敏译，清华大学出版社，2014
 - ✓ 《大话数据结构》，程杰著，清华大学出版社，2011
 - ✓ 《数据结构(C语言版)》，严蔚敏 吴伟民 编著，清华大学出版社，2018



教学网站

◆教学网站（课件、作业、答疑和考试）

✓直接访问：<https://judge.buaa.edu.cn/>

✓VPN访问：VPN(<https://e.buaa.edu.cn/>)，

选择VPN2或VPN3，登录后进入“6系教辅平台”（如图）

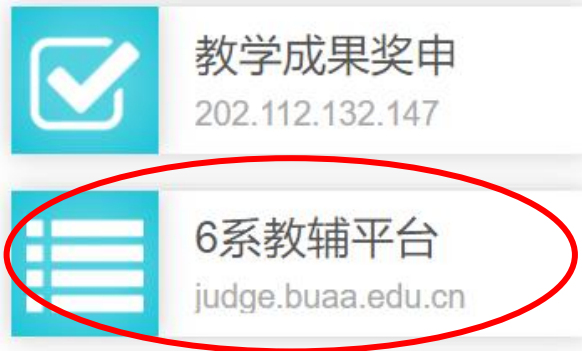
✓期中期末考试时要运行考试客户端（禁外网程序），

该客户端仅支持Windows系统，请同学们提前知悉！

◆北航统一认证登录

◆选择课程：数据结构与程序设计

教务教学





线上课程:

◆智慧树慕课：数据结构与程序设计

✓ <https://coursehome.zhihuishu.com/courseHome/1000103811>



在线学堂

数据结构与程序设计

工学 (08) 计算机类 (0809)



课程介绍

[申请学校选课](#)

本课程在长期实践中，提出了“学为中心，问题驱动，知行合一”的教学理念，构建了“一体两翼三课堂”的课程体系，创新了“问题驱动，教师精讲；实践驱动，知行合一；思维驱动，授之以渔；平台驱动，精准质控”的教学方法；设计了以复杂工程问题的程序设计方法为主线，以计算报国情怀、自

[查看更多>](#)

学分 2.0 学时 31.0 见面课 6次

教师 晏海华、郝爱民、刘祥龙、孙自安、李莹、谭火彬、蒲菊华、盛浩、尚涛

学校 北京航空航天大学

书名 数据结构教程 (第3版) 作者 唐发根

出版社 北京航空航天大学出版社 出版月份 2017-06

ISBN 978-7-5124-2432-6

[了解更多](#)



关于本课程：数据结构与程序设计（信息类）

- ◆总学时：32上课学时+32上机学时（3学分）
- ◆计算机、软件工程、网络安全等学科最重要专业基础课之一
- ◆后续专业课程学习的必要知识与技能准备
 - ✓算法分析与设计课程的先修课
 - ✓编译技术要使用栈、散列表及语法树
 - ✓操作系统中使用队列（优先队列）、存储管理表及目录树
 - ✓数据库系统运用线性表、多链表、及索引树
 - ✓...
- ◆成为一名专业程序员应具备的基本技能
 - ✓大部分公司技术笔试或面试的必考内容



教学内容安排

- ◆第0章 绪论 (1)
- ◆第1章 程序设计基础 (5)
- ◆第2章 线性表 (4)
- ◆第3章 堆栈和队列 (4)
- ◆第4章 树和二叉树 (6)
- ◆第5章 查找 (3)
- ◆第6章 排序 (3)
- ◆第7章 图 (6)



上机作业安排 (20%)

- ◆ **普通作业 (15%)**：期限内完成，主要考查对单一知识点的应用能力
 - ✓ 共7套（基础1、基础2、线性表、栈和队、树、查找与排序、图）
 - ✓ 每套有一组选择填空概念题，多个应用编程题
- ◆ **综合性能 (Project) 作业 (5%)**，主要考查学生对数据结构综合应用的能力；有3道要求完全一样的题目，但测试数据的规模不一样
 - ✓ **小测试数据**题评判要求 (2.5分)：正确占100%，不考查性能。为了方便调试，给出了跟测试数据基本一样的样例数据
 - ✓ **大测试数据**题评判要求 (2分)：正确占20%，性能占80%。即在给定时间 (100秒) 内正确完成，得20%分，性能部分分数以运行最快的前10%程序为基准 (其为满分)，依次计算得分，运行结果不正确或在给定时间内没有运行结束，则不得分
 - ✓ **期末评判**题 (0.5分)：平时只提交，不评判，课程结束后评判。测试数据比大测试数据题要大一些，只考查正确性 (可扩展性)，不考查性能



上机作业安排（续）

◆应在**规定时间**内提交；

- ✓每套普通作业大约开放2~4个星期
- ✓综合性能题大约4月初开放，期末考试前关闭

◆按题目要求提交，比如：输入、输出数据格式，提交文件名称等等

◆**严禁抄袭**，编程作业查出抄袭要处理！（作业系统有查重功能，务必重视！）

- ✓不能拷贝提交他人源代码（提交同学的代码）
- ✓不能下载提交网上源代码（提交网上的代码）
- ✓不要将自己的源码拷贝给他人（可能被抄袭）



学习目标和方法

- ◆目标：培养学生利用所学知识解决问题的能力，包括解决没有遇到过的问题的能力（即程序设计能力）
- ◆方法：
 - ✓中国著名桥梁专家、工程教育家茅以升先生提出了：“习而学”、“做中学”（Learning by Doing）的工程教育思想

上机实践(编程)! (Try!!!)





提纲：绪论

◆ 0.1 课程简介

◆ 0.2 数据结构基础

- ✓ 0.2.1 数据结构基本概念

- ✓ 0.2.2 数据结构的主要内容

◆ 0.3 算法基础



软件中的数据结构

软件 = ? 程序 = ?

软件 = 程序 + 文档

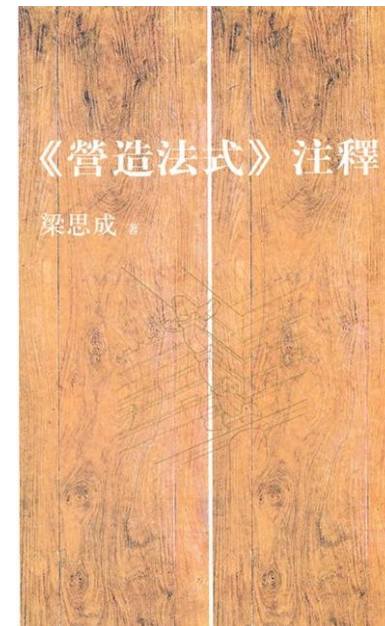
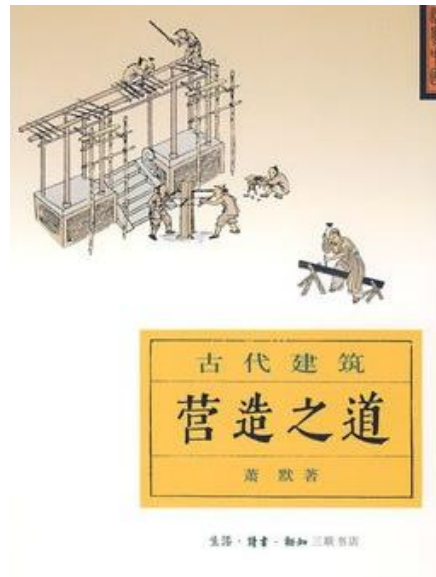
程序 = 数据结构 + 算法
(图灵奖得主N.Wirth(沃斯))

好数据结构 + 好算法 + 好风格 = 好程序

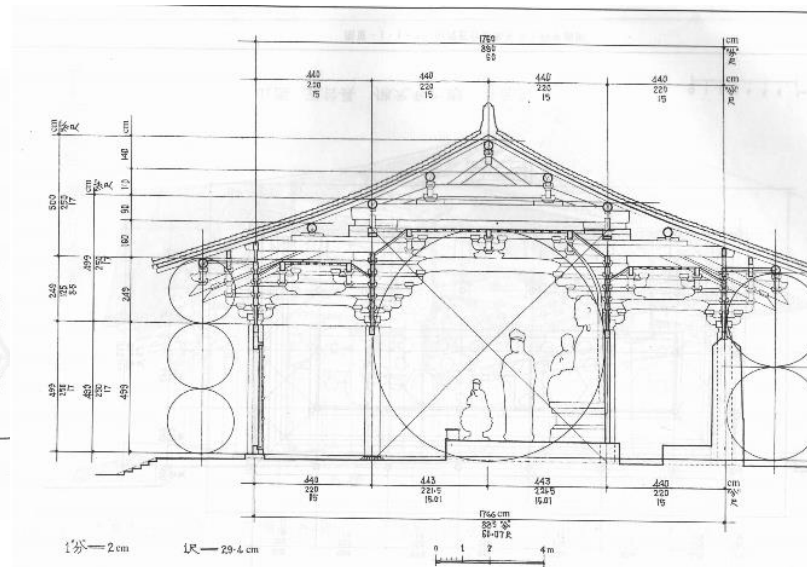
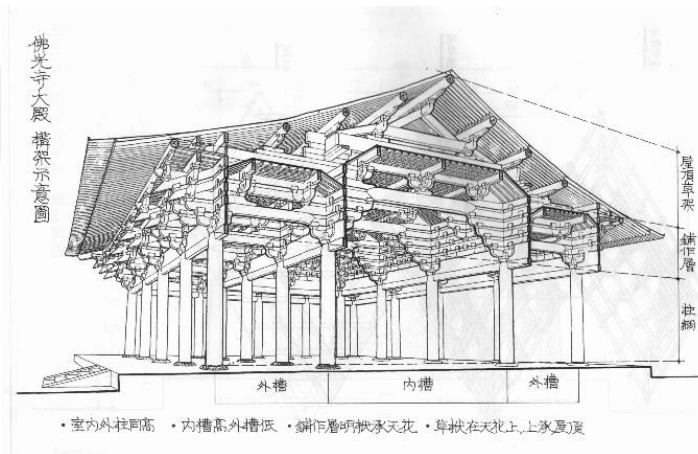


程序设计与数据结构

程序设计：如何使用程序设计语言（工具）来解决问题



数据结构：研究数据的构造方法





什么是数据结构

数据

描述客观事物的数字、字符以及一切能够输入到计算机中，并且能够被计算机程序处理的符号的集合

数据元素

数据集合中的一个一个的元素

数据对象

具有相同特性的数据元素的集合

数据结构

数据元素之间具有的关系（相互之间存在一种或多种特定关系的数据元素的集合）



示例：数据、数据元素、数据结构

1

(25, 78, 36, 100, 28, 45)

数列

2

('A', 'B', 'C', ..., 'Z')

字母表

3

学号 姓名 性别 年龄 其他

99001	张三	女	17	
99002	李四	男	16	
99003	王五	女	18	
99004	周六	女	17	
⋮	⋮	⋮	⋮	⋮
99035	刘末	男	19	

数据对象



数据结构的正式定义

- ◆ 数据元素之间的联系称之为**结构**，**数据结构**就是具有结构的数据元素的集合
- ◆ 数据结构是一个二元组：Data-Structure = (D, R) ，其中：
 - D是数据元素的有限集合（即某一数据对象）
 - R是D上的关系集合



数据结构研究的内容

逻辑结构 + **存储结构** + **算法**

◆数据的**逻辑结构**

- ✓通过**抽象方法**研究被处理数据间存在何种**逻辑关系**，即逻辑结构
- ✓有两大类：线性结构（线性表、数组、栈、字符串等）和非线性结构（树、图等）

◆数据的**物理（存储）结构**

- ✓研究逻辑关系在**计算机内部如何表示**，即存储结构/物理结构
- ✓如：线性存储结构、链式存储结构、索引结构和散列结构

◆数据的**操作（算法）**

- ✓研究在数据各种结构的基础上如何**对数据实施有效的操作或处理**
- ✓如：创建、清除、插入、删除、搜索、更新、访问、遍历等



逻辑结构：线性结构和非线性结构

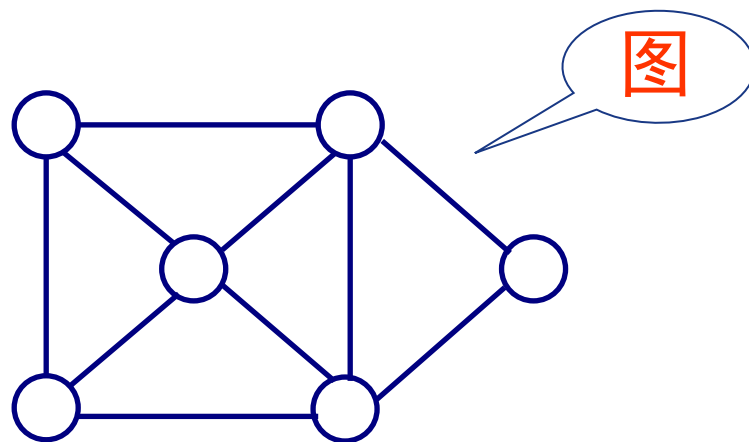
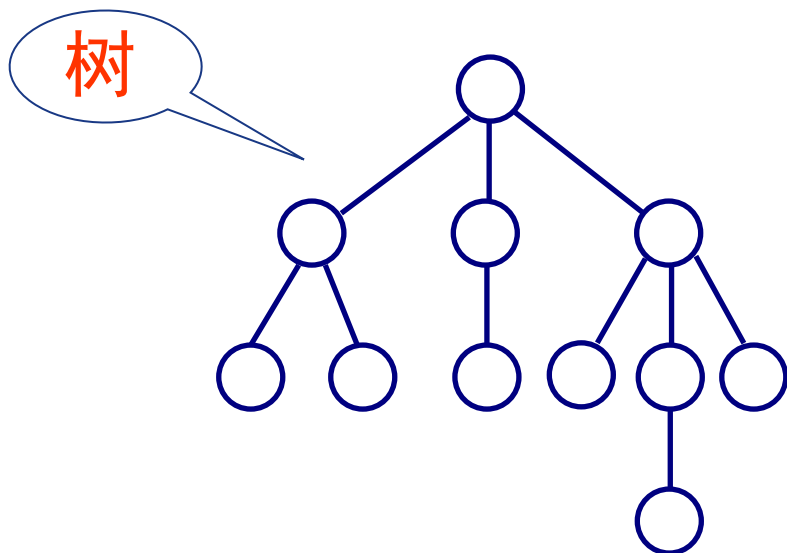
◆ 1. 线性结构

✓如：线性表、堆栈、队列、串、文件等



◆ 2. 非线性结构

✓如：树、二叉树、图、集合等



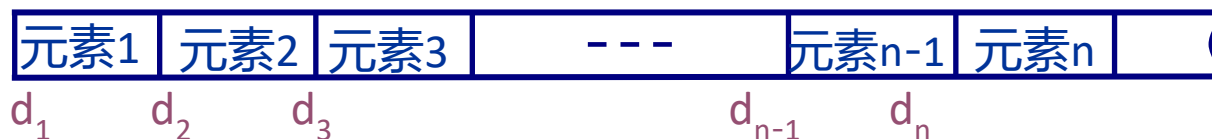


物理（存储）结构：顺序、链式

◆ 1. 顺序存储结构

✓ 用一组**地址连续**的存储单元依次存放数据元素，数据元素之间的逻辑关系通过元素**地址直接**反映，即：数据间的逻辑关系与物理关系是一致的

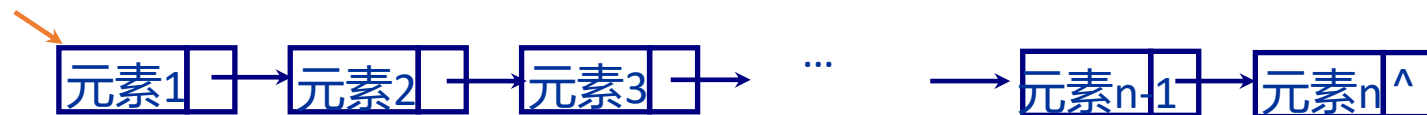
✓ 如：数组



◆ 2. 链式存储结构

✓ 用一组**地址任意**的存储单元依次存放数据元素，数据元素间的逻辑关系通过**指针间接**地反映

✓ 每个数据节点由两部分组成：(1)数据本身（数据字段）；(2)指针：存放后续结构的地址（指针字段）





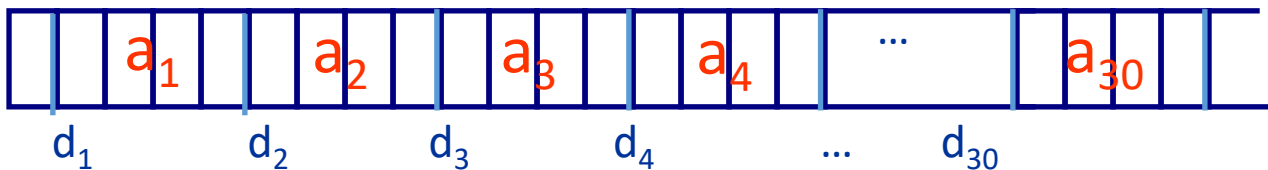
示例：理解逻辑结构和物理结构

逻辑结构： 线性结构（线性表）

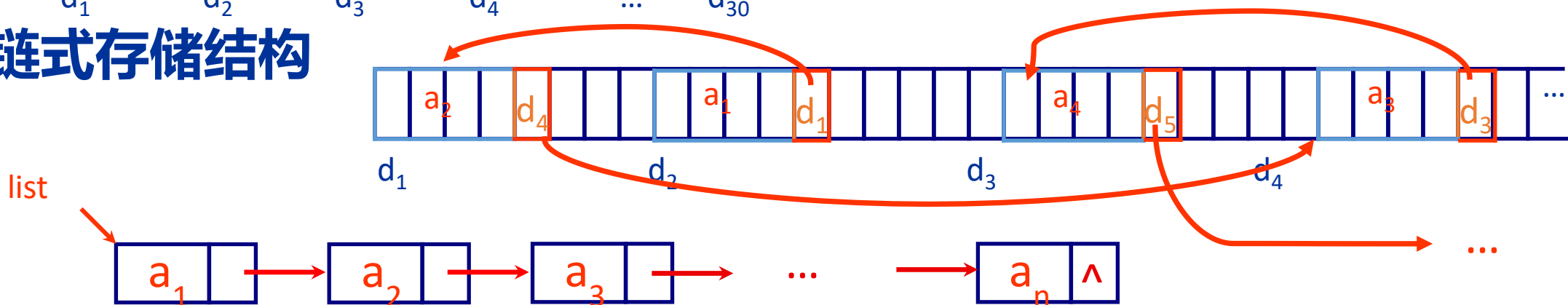
	姓 名	性 别	民 族	年 龄	其 他
a_1	刘晓光	男	汉	16	...
a_2	马广生	男	回	17	...
a_3	王 民	男	壮	19	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
a_{30}	张淑华	女	汉	24	...

存储结构：

1. 顺序存储结构



2. 链式存储结构





物理（存储）结构：索引和散列（后续讨论）

◆ 3. 索引存储结构

- ✓原理：利用数据元素的**索引关系**来确定数据元素的存储位置，由数据元素本身与索引表两部分组成
- ✓特点：诸如查找、插入和删除等操作的时间效率较高，但存储空间开销较大

◆ 4. 散列（哈希）存储结构

- ✓原理：通过事先准备好的**散列函数**与处理冲突的方法来确定数据元素的存储位置
- ✓特点：诸如查找、插入和删除等操作的时间效率较高,主要缺点是确定好的散列函数比较困难



操作（算法）

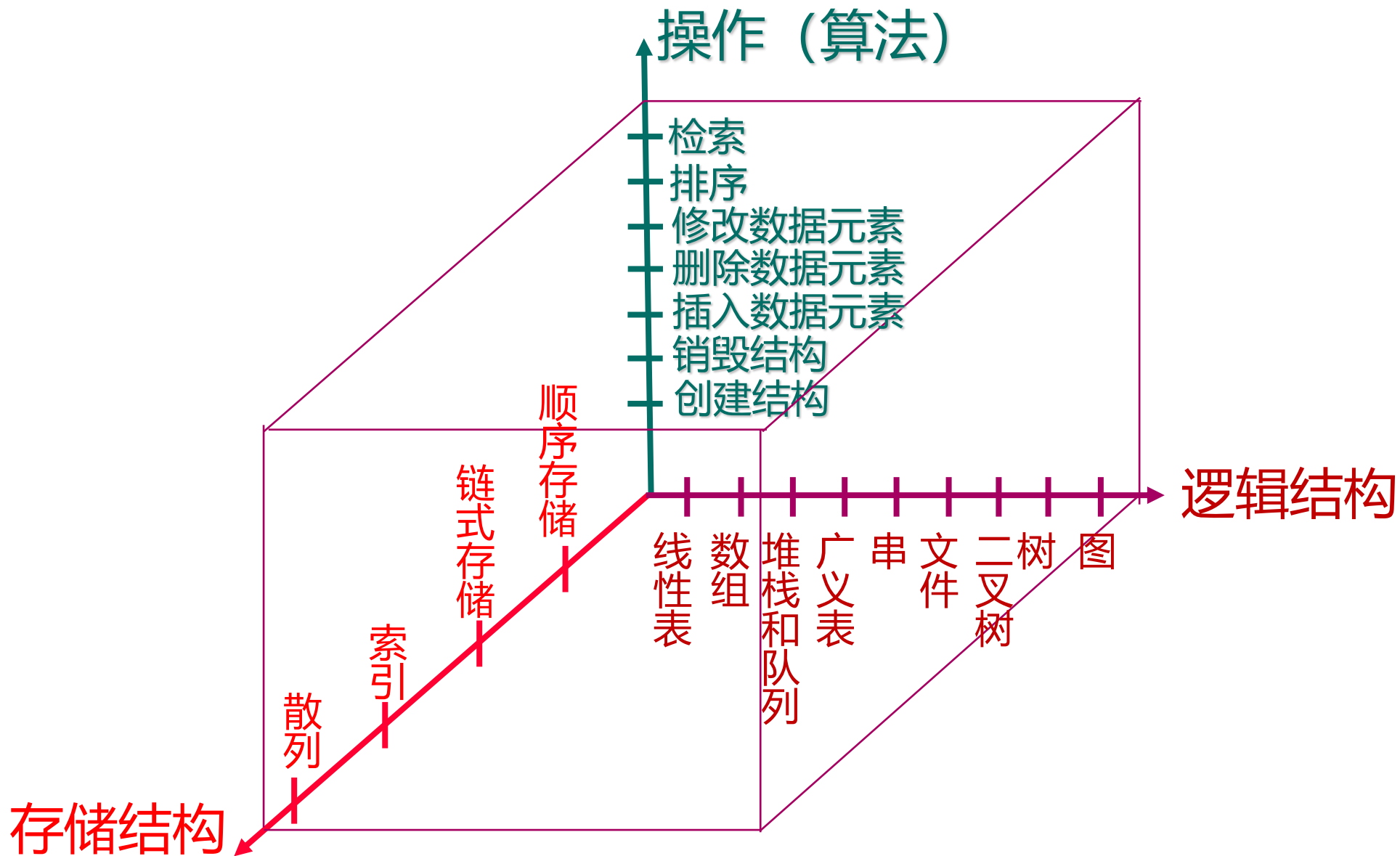
◆操作：对具有某种逻辑结构的数据所实施的一系列操作（算法）

◆基本操作：

- ✓ **创建**：构造具有某种逻辑结构的数据集，如构造一个线性表、链表、树和图等
- ✓ **检索**：在已有数据集中查找某一指定元素
- ✓ **遍历**：访问数据集中所有元素
- ✓ **插入**：在已有数据集中插入一指定元素
- ✓ **删除**：在已有数据集中插入一指定元素
- ✓ **修改**：在已有数据集中修改一个指定元素
- ✓ **排序**：对一数据集中元素按照某一顺序进行排列
- ✓ ...



数据结构的基本问题空间





提纲：绪论

- ◆ 0.1 课程简介
- ◆ 0.2 数据结构基础
- ◆ 0.3 算法基础
 - ✓ 0.3.1 算法的基本概念
 - ✓ 0.3.2 算法分析初步



算法

- (1) 算法是用来解决某个特定问题的指令的集合
- (2) 算法是由人们组织起来准备加以实施的一系列有限的基本步骤
- (3) 算法是一组解决问题的清晰指令，它能够对符合一定规范的输入，在有限的时间内获得所需要的输出



算法的性质

◆ 一个完整的算法，应具有下面五个基本特征

- ✓ **输入**：由算法的外部提供 $n \geq 0$ 个有限量作为算法的输入
- ✓ **输出**：由算法的内部提供 $n > 0$ 个有限量作为算法的输出
- ✓ **有穷性**：算法必须在有限的步骤内能够结束
- ✓ **确定性**：组成算法的每一条指令必须有清晰明确的含义
- ✓ **有效性**：算法的每一条指令必须具有可执行性



算法的描述方法

- ◆ 1. 采用自然语言来描述
- ◆ 2. 采用图形化符号来描述
 - ✓ 程序流程图：表示顺序、选择、循环结构
 - ✓ UML活动图
- ◆ 3. 采用伪代码来描述
 - ✓ 类似于某种高级语言，但又没有严格的语法和格式要求
 - ✓ 如：SPARKS语言（一种类Pascal语言）
- ◆ 4. 采用某种程序设计语言来描述



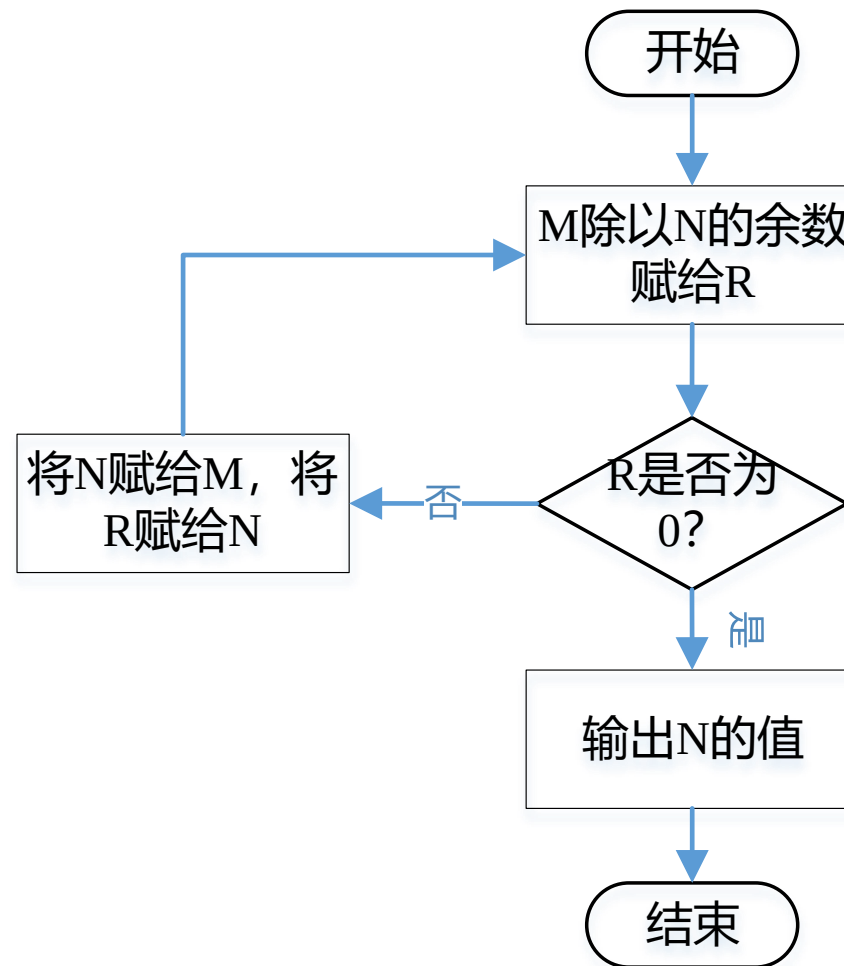
示例：算法的描述方法

问题

求两个正整数M与N的最大公因子

- (1) M除以N, 将余数赋给中间变量R;
- (2) 判断余数R是否等于零?
 - a) 若R等于零, 求得的最大公因子为当前N的值, 算法到此结束。
 - b) 若R不等于零, 则将N赋给M, 将R赋给N, 重复步骤(1)和(2)。

```
int comfactor( int m, int n ){  
    int r;  
    while( 1 ){  
        r = m % n;  
        if( r == 0 ) return n;  
        m = n;  
        n = r;  
    }  
}
```





算法分析

- ◆ 算法分析是指对算法质量（效率）优劣的评价，其目的是为了改进算法的质量
- ◆ 算法分析的前提是首先要保证算法的**正确性**，在此基础上，可以从多个方面分析算法的性能（优劣）
- ▲ 依据算法编写的程序在计算机中**运行时间**多少的度量，称之为 **时间复杂度**
反映算法运行的快慢
- ▲ 依据算法编写的程序在计算机中**运行时占存储空间**多少的度量，称之为 **空间复杂度**
反映算法需要的额外空间的多少
- ▲ 其他方面，如：算法可读性、可移植性以及易测试性的好坏（**代码的设计质量**）



时间复杂度

◆程序在计算机中运行时间的多少与诸多因素有关，主要有：

1. 问题的规模

几乎所有算法的时间效率都与问题的规模有关

2. 编译程序功能的强弱以及所产生的机器代码质量的优劣

3. 机器执行一条指令的时间长短

4. 程序中那些基本语句的执行次数

重点

对算法运行时间贡献最大的语句

相关



时间复杂度的表示法：频度统计法

◆频度分析法：以语句执行的次数的多少作为算法的时间度量的分析方法

✓一条语句的频度是指该语句被执行的次数，而整个算法的频度是指算法中所有语句的频度之和

```
#define M 1000
void MATRIX(int A[ ][M],int B[ ][M],int C[ ][M],int n)
{
    int i, j, k;
    for(i=0;i<n;i++)
        for(j=0;j<n;j++){
            C[i][j]=0;
            for(k=0;k<n;k++)
                C[i][j]=C[i][j]+A[i][k]*B[k][j];
        }
}
```

语句频度

/* 执行 n^2 */

/* 执行 n^3 */

算法的频度：

$$t(n) = n^2 + n^3$$



频度统计法的分析

◆假设算法A的语句频度为 $2n^2$ ，算法B的频度为 $3n+1$ ，算法C的语句频度为 $2n^2+3n+1$ ，则有以下表：

次数	算法 A ($2n^2$)	算法 B ($3n+1$)	算法 C ($2n^2+3n+1$)
$n = 1$	2	4	6
$n = 2$	8	7	15
$n = 5$	50	16	66
$n = 10$	200	31	231
$n = 100$	20 000	301	20 301
$n = 1,000$	2 000 000	3 001	2 003 001
$n = 10,000$	200 000 000	30 001	200 030 001
$n = 100,000$	20 000 000 000	300 001	20 000 300 001
$n = 1,000,000$	2 000 000 000 000	3 000 001	200 000 3000 001

从中可以看出：

判断一个算法的效率时，算式中的常数和和其它次要项通常可以忽略，而更应该关注主要项（**最高阶项**）的阶数



时间复杂度的表示法：大O表示法

当且仅当存在正整数 c 和 n_0 ,使得 $t(n) \leq cg(n)$ 对所有的 $n \geq n_0$ 成立,则称函数 $t(n)$ 与 $g(n)$ 同阶,或者说, $t(n)$ 与 $g(n)$ 同一个数量级, 记作

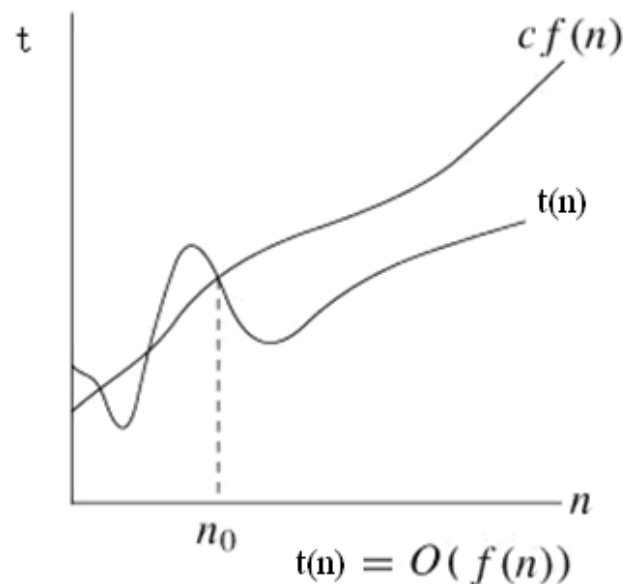
$$t(n) = O(g(n))$$

称上式为算法的渐近时间复杂度, 或称该算法的时间复杂度为 $O(g(n))$ 。
其中, n 为问题的规模(大小)的量度

当问题的输入规模 n 趋于无穷大时, 算法的运行时间表现出固定的增长次数

$$t(n) = n^3 + n^2 \quad g(n) = n^3$$

称算法的时间复杂度为 $O(n^3)$





大O (Order) 表示法

- ◆大O表示法关注的是问题规模 n 增长时，算法执行次数增长的数量级。因此，可以只关注算法中**基本语句**执行频度,不必对算法的每一个步骤都进行详细的分析

```
i=1;           (1)
while (i<=n)    (2)
    i=i*2;      (3)
```

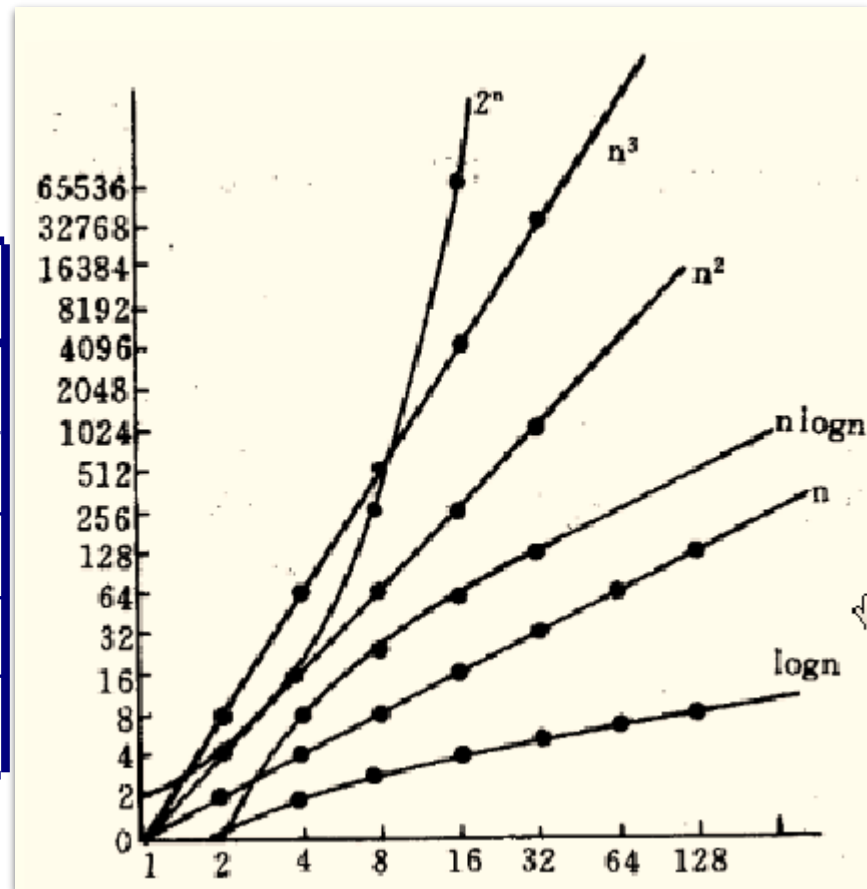
可只关注语句(3)的频度，设为 $t(n)$,则有: $2^{t(n)} \leq n$
即: $t(n) \leq \log_2 n$, 取最大值 $t(n) = \log_2 n$
时间复杂度为: $O(\log_2 n)$



常见的时间复杂度

◆对于算法分析具有重要意义的常见函数值

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	3.3×10^1	10^2	10^3	10^3	3.6×10^6
100	6.6	10^2	6.6×10^2	10^4	10^6	1.3×10^{30}	9.3×10^{157}
1000	10	10^3	1.0×10^4	10^6	10^9		
10000	13	10^4	1.3×10^5	10^8	10^{12}		
...							



常见的时间复杂度:

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

$O(1)$: 表示算法的复杂度为常量, 不随问题规模 n 的大小而改变



示例：求时间复杂度

◆求两个n阶方阵的相加 $C=A+B$ 的算法如下,分析其时间复杂度

```
#define MAX 20 /*定义最大的方阶*/  
void matrixadd(int n, int A[MAX][MAX], int B[MAX][MAX], int C[MAX][MAX])  
{  
    int i, j;  
    for (i = 0; i < n; i++)  
        for (j = 0; j < n; j++)  
            C[i][j] = A[i][j] + B[i][j];  
}
```

◆算法中的基本运算是两重循环中最深层的语句： $C[i][j]=A[i][j]+B[i][j]$

✓分析频度： $T(n)=\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}1=\sum_{i=0}^{n-1}n=n\sum_{i=0}^{n-1}1=n*n=n^2$

✓时间复杂度： $O(n^2)$



示例：求时间复杂度（续）

◆分析以下算法的时间复杂度

```
int fun(int n)
{
    int i, j, k, s;
    s = 0;
    for (i = 0; i <= n; i++)
        for (j = 0; j <= i; j++)
            for (k = 0; k <= j; k++)
                s++;
    return (s);
}
```

◆ 该算法的基本操作是语句s++，频度：

$$T(n) = \sum_{i=0}^n \sum_{j=0}^i \sum_{k=0}^j 1 = n^3$$

✓ 时间复杂度：O(n³)



示例：求时间复杂度（续）

◆分析以下算法的时间复杂度

```
void func(int n)
{
    int i = 0, s = 0;
    while (s < n)
    {
        i++;
        s = s + i;
    }
}
```

- ◆ 对于while循环语句，设执行次数为 m ， i 从0开始递增1，直到 m 为止，有： $s = 0 + 1 + 2 + \dots + m-1 = m(m-1)/2$ ，并满足 $s = m(m-1)/2 < n$ ，则有 $m \leq \sqrt{n}$
- ◆ 时间复杂度： $O(\sqrt{n})$



示例：求时间复杂度（续）

◆调用算法fun(a,n,0)，求其时间复杂度

```
void fun(int a[], int n, int k) //数组a共有n个元素
{
    int i;
    if (k == n - 1)
        for (i = 0; i < n; i++)
            printf("%d\n", a[i]);
    else
    {
        for (i = k; i < n; i++)
            a[i] = a[i] + i * i;
        fun(a, n, k + 1);
    }
}
```

◆ 设fun(a,n,0)时间复杂度为 $T(n)$ ，fun(a,n,k)执行时间为 $T_1(n,k)$ ，由fun()算法可知：

✓ 当 $k=n-1$ 时： $T_1(n, k)=n$

✓ 其它情况： $T_1(n, k)=(n-k)+T_1(n,k+1)$

◆ 则：

$$T(n)=T_1(n,0)=n+T_1(n,1)=n+(n-1)+T_1(n,2)=\dots$$

$$=n+(n-1)+\dots+2+T_1(n,n-1)=n+(n-1)+\dots+2+n=n^2$$

◆ 调用fun(a,n,0)的时间复杂度： $O(n^2)$



时间复杂度：最坏情况和平均情况

- ◆ 对算法的分析，一种是**平均时间复杂度**；另一种是**最坏时间复杂度**
 - ✓ 一般在没有特殊说明的情况下，都是指最坏时间复杂度
- ◆ 最坏情况运行时间是一种保证，那就是运行时间将不会再坏了
 - ✓ 在实际应用中，这是一种最重要的需求。如机载软件关键软件中必须要做最坏情况时间分析
- ◆ 平均运行时间是所有情况中最有意义的，因为它是期望的运行时间



延伸学习* - 算法空间复杂度

- ◆ 算法空间复杂度通过计算算法运行时所需的存储空间来实现
- ◆ 算法空间复杂度是衡量算法效率的另一个重要指标
- ◆ 程序设计时，经常会用空间来换时间



小结

- ◆ 了解课程的总体安排
 - ✓ 认识课程的定位，理解课程要求和目标
- ◆ 掌握数据结构的基本概念
 - ✓ 逻辑结构、物理（存储）结构和算法
- ◆ 掌握算法的基本概念，能够分析算法的时间复杂度
 - ✓ 算法的性质、描述方法
 - ✓ 时间复杂度
 - ✓ 空间复杂度